sercon

User Manual

Version 0.8.0

2026-05-26

Repository · https://github.com/codedeviate/sercon

License · MIT

sercon manual

Long-form reference for the `pkg/scriptengine` library, the `sercon` CLI, the built-in script API, and the JavaScript runtime surface that `scriptengine` exposes via goja and goja_nodejs. Examples are runnable — save snippets as `.ts` files and execute with `sercon file.ts` unless otherwise noted.

Table of contents

1. Overview
2. Quickstart
3. Library API — `pkg/scriptengine`
4. CLI — `sercon`
5. Reserved globals (script surface)
6. JavaScript runtime built-ins (goja)
7. Async runtime additions (goja_nodejs)
8. TypeScript support
9. Top-level `await`
10. Module resolution
11. Timeouts and cancellation
12. Error semantics
13. Type generation (.d.ts)
14. Limitations and gotchas

---

1. Overview

`sercon` is a Go program that runs TypeScript files. It's built from two parts:

- `pkg/scriptengine` — a library you embed in your own Go code. You register Go-callable bindings, then call `Run` / `RunFile` to execute a `.ts` script that talks to them.
- `cmd/sercon` — a thin CLI built on the library, useful for ad-hoc test scripts and as a working example of every binding kind.

The runtime is pure Go: goja is the JS engine, esbuild (used as a Go library) is the TS→JS transpiler, and goja_nodejs provides Promises, `setTimeout`, `console`, and CommonJS `require`. No cgo, no Node.

2. Quickstart

Install:

```
go install github.com/codedeviate/sercon/cmd/sercon@latest
```

Run one of the bundled examples:

```
sercon examples/scripts/smoke.ts examples/scripts/async.ts
```

…or run them all via `make demo`. `examples/scripts/` ships a runnable `.ts` (or `.tsx`) file per feature area — `hash.ts`, `strings.ts`, `path-and-time.ts`, `default-export.ts`, `tsx-demo.ts`, and the original `smoke.ts` / `async.ts` / `hang.ts` (`hang.ts` is the timeout demo and intentionally exits non-zero).

Generate a declaration file for editor autocomplete:

```
sercon --emit-dts d.ts
```

Embed in your own Go program:

```
eng := scriptengine.New(scriptengine.Options{
    Timeout:    5 * time.Second,
    ScriptRoot: "./scripts",
})
eng.Register("upper", strings.ToUpper)
val, err := eng.RunFile(ctx, "scripts/main.ts")
```

3. Library API — `pkg/scriptengine`

`Engine`, `Options`, `New`

```
type Options struct {
    Timeout        time.Duration // wall-clock limit per Run; 0 disables
    ScriptRoot     string        // base for require/import resolution
    DisableConsole bool          // turn off the goja_nodejs console module
    Verbose        io.Writer     // diagnostic traces, prefixed "[sercon] "
    ModuleLoader   func(candidatePath string) (source string, found bool,
err error)
}

func New(opts Options) *Engine
```

`ModuleLoader`, when non-nil, is consulted for every require/import candidate path **before** the filesystem — the hook for embedders that want to serve modules from somewhere other than disk (an in-memory FS, a network source, an embedded bundle). goja probes several candidates per specifier (`./x`, `./x.ts`, `./x/index.ts`, …); the engine also tries the bare path plus the usual extension fallbacks (`.ts` / `.tsx` / `.js` / `.mjs` / `.cjs` / `.json`) against the loader, so a loader can match on a plain suffix. Return `(source, true, nil)` to serve the module (transpiled when the path ends in `.ts` / `.tsx`, exactly like a disk read); `("", false, nil)` to fall through to the filesystem; `("", false, err)` to abort resolution.

```
eng := scriptengine.New(scriptengine.Options{
    ModuleLoader: func(path string) (string, bool, error) {
        if strings.HasSuffix(path, "greeting.ts") {
            return `export const hi = (n: string) => "hi " + n;`, true, nil
        }
        return "", false, nil // fall through to disk
    },
})
```

`Engine` is the host of all registered bindings. Construct it once, then call `Run` / `RunFile` as many times as needed — each call gets a *fresh* `*goja.Runtime` so script state never leaks between invocations.

`ScriptRoot` defaults to the working directory at `New` time if left empty. Both `Timeout` and `ScriptRoot` can be overridden on a per-Run basis only by constructing a fresh `Engine`; see Out-of-scope for the per-Run override on the roadmap.

Registering bindings

Function	Use when…
`Register(name, value)`	The binding is a pure function, struct, or primitive that doesn't need access to the runtime.
`RegisterNamespace(name, members map[string]any)`	You want to expose `name.x`, `name.y` without defining a Go struct.
`RegisterConstructor(name, ctor)`	The binding is a Go function whose return value should be treated as a class in the emitted `.d.ts`. (Runtime semantics today are identical to `Register`; the d.ts emitter is the only differentiator.)
`RegisterFactory(name, factory func(vm, loop) any)`	The binding needs the per-Run `*goja.Runtime` or `*eventloop.EventLoop` in scope (typical for Promise-returning I/O).

Function	Use when…
`RegisterNamespaceFactory(name, factory)`	Same, but the factory returns the full member map.

Example — synchronous function binding:

```
eng.Register("greet", func(name string) string { return "hi " + name })
```

Example — namespace with a sync member:

```
eng.RegisterNamespace("math2", map[string]any{
    "square": func(n float64) float64 { return n * n },
    "pi":     3.14159,
})
```

Example — async (Promise-returning) binding via `PromisifyAsync`:

```
eng.RegisterFactory("httpGet", func(vm *goja.Runtime, loop
*eventloop.EventLoop) any {
    return scriptengine.PromisifyAsync(vm, loop,
        func(ctx context.Context, call goja.FunctionCall) (map[string]any,
error) {
            url := call.Argument(0).String()
            resp, err := http.Get(url)
            if err != nil {
                return nil, err
            }
            defer resp.Body.Close()
            body, _ := io.ReadAll(resp.Body)
            return map[string]any{"status": resp.StatusCode, "body":
string(body)}, nil
        })
})
```

`Run`, `RunFile`

```
func (e *Engine) Run(ctx context.Context, name, source string, opts
...RunOption) (goja.Value, error)
func (e *Engine) RunFile(ctx context.Context, path string, opts
...RunOption) (goja.Value, error)
```

`Run` executes `source` as an entry-script TS. `name` is used in stack traces. The returned value is currently always `undefined` — top-level expression capture is on the backlog.

Both methods respect `Options.Timeout` *and* `ctx` cancellation, whichever fires first; the resulting error is either `ErrScriptTimeout`, `ctx.Err()`, or the underlying JS exception.

Per-Run options

```
type RunOption func(*runConfig)

func WithScriptRoot(dir string) RunOption
```

Reuse a single Engine across many scripts that each live in their own directory by passing `WithScriptRoot(dir)` to `Run` / `RunFile`. The override applies only to this call; `Options.ScriptRoot` is used for every other Run.

```
_, _ = eng.Run(ctx, "main.ts", source,
scriptengine.WithScriptRoot("/path/to/run42"))
```

```
func WithArgs(args []string) RunOption
```

Set the per-script argument vector. The script reads it as `runtime.argv`, a global with Node/Bun layout: `argv[0]` is `Options.ProgramName` (defaults to `filepath.Base(os.Args[0])`; the CLI sets it to `"sercon"`), `argv[1]` is the running script's path, and the `WithArgs` values follow from index 2. The global is always present — with no args it is just `[programName, scriptPath]`.

```
_, _ = eng.Run(ctx, "main.ts", source, scriptengine.WithArgs([]string{"--
port", "8080"}))
// inside the script: runtime.argv === [ProgramName, "/abs/main.ts", "--
port", "8080"]
```

`Reset`

```
func (e *Engine) Reset()
```

Clears every registered binding. Useful when a long-lived Engine is reused across unrelated batches of scripts that each want a clean global namespace. **Not safe to call concurrently with Run / RunFile.**

`WriteTypes`

```
func (e *Engine) WriteTypes(w io.Writer) error
```

Emits a `.d.ts` describing the registered surface. See section 13. Type generation for what the mapping looks like.

`SetDocs` and `SetMemberDocs`

```
func (e *Engine) SetDocs(path, doc string)
func (e *Engine) SetMemberDocs(namespace string, docs map[string]string)
```

Attach JSDoc strings to registered bindings so the emitted `.d.ts` grows readable editor hover. `SetDocs` takes a dotted path — either a bare top-level name (`"greet"`, `"http"`) or `"namespace.member"` for namespace members (`"http.get"`, `"exec.shell"`); `SetMemberDocs` is the convenience for documenting many members of a namespace at once (the namespace itself can still be documented separately via `SetDocs`).

Multi-line docs are written as `\n`-separated strings; the emitter expands them into a standard `*`-prefixed JSDoc block. Single-line docs collapse to `/** … */`. Calling `SetDocs` with an empty string removes any previous doc for that path. Bindings without a doc entry get no JSDoc block at all — the emitter doesn't insert empty `/** */` placeholders.

`SetDocs` may be called before or after the matching `Register…` call; docs are looked up by name at emit time. Like the registration methods, `SetDocs` must not race with a `Run` / `WriteTypes` / `Reset` on the same engine.

```
eng.Register("greet", func(name string) string { return "hi " + name })
eng.SetDocs("greet", "Greet someone by name.")

eng.RegisterNamespace("math2", map[string]any{
    "pi":      3.14159,
    "squared": func(n float64) float64 { return n * n },
})
eng.SetDocs("math2", "Tiny math helpers.")
eng.SetMemberDocs("math2", map[string]string{
    "pi":      "Circumference / diameter ratio.",
    "squared": "Multiply a number by itself.",
})
```

`PromisifyAsync[T]` and `AsyncBinding`

```
type AsyncBinding struct {
    Func         func(goja.FunctionCall) goja.Value
    TSReturnType string
}

func PromisifyAsync[T any](vm *goja.Runtime, loop *eventloop.EventLoop,
    work func(ctx context.Context, call goja.FunctionCall) (T, error),
) AsyncBinding
```

`PromisifyAsync` turns blocking Go work into a JS Promise. It launches the work in a goroutine, parks a `SetTimeout` sentinel so `loop.Run` doesn't return early, and schedules the

resolve/reject back onto the event loop. **Required** for any Promise-returning binding — `RunOnLoop` alone is not counted as a live job by the event loop.

`PromisifyAsync` returns an `AsyncBinding` carrier rather than the bare callback. The engine unwraps it to `.Func` at `vm.Set` time so goja's special-case detection of `func(goja.FunctionCall) goja.Value` host-callbacks still fires; the `.d.ts` emitter reads `.TSReturnType` to emit `Promise<T>` (mapped from the generic `T` via reflect at construction time) instead of the previous `unknown`. Hosts assigning the result into a `map[string]any` for `RegisterNamespace` / `RegisterNamespaceFactory` get the unwrap recursively too — no manual work required.

Version

```
import "github.com/codedeviate/sercon/pkg/scriptengine"

fmt.Println(scriptengine.Version) // "0.1.0"
```

Bumped in lockstep with the git tag.

4. CLI — `sercon`

```
sercon [flags] <script.ts> [script.ts ...]
sercon [flags] -                # read entry script from stdin
sercon --examples | --help | --version
```

Flag	Purpose
`-timeout DURATION`	Wall-clock limit per script (default `10s`; `0` disables).
`-root DIR`	Override the script root for `require` / `import` resolution.
`-emit-dts PATH`	Write the example bindings' `.d.ts` to `PATH` and exit.
`-v`	Verbose: trace the rewritten entry-script JS and each module resolution to stderr; also print duration on failure.
`-h`, `--help`	In-depth colorized help.
`--examples`	In-depth colorized walkthrough of every feature.
`--version`	Print the engine version (plus goja/esbuild build-info versions).
`--watch`	Re-run on file change under the script root. Debounced 150 ms. Ctrl-C exits.

Each positional argument is either a path to a `.ts` / `.tsx` file or `-` to read an entry script from standard input:

```
echo 'runtime.log(1 + 2);' | sercon -
sercon prelude.ts -                # prelude then stdin
```

Passing arguments: `--` and `runtime.argv`

Everything after a standalone `--` is the script argument vector, exposed to every script via `runtime.argv`:

```
sercon run.ts -- --port 8080 hello
```

`runtime.argv` uses the Node/Bun layout `[programName, scriptPath, ...userArgs]` — `argv[0]` is `"sercon"`, `argv[1]` is the path of the script currently executing, and the post-`--` arguments start at index 2 (so `runtime.argv.slice(2)` is just your args). All scripts in one invocation share the same `--` tail, but each sees its own path at `argv[1]`. The global is always present; with no `--` it is just `[programName, scriptPath]`.

```
const args = runtime.argv.slice(2);   // ["--port", "8080", "hello"]
```

Exit codes are distinct per failure type so shells can react sensibly. When several scripts run, the highest applicable code wins:

Code	Meaning
`0`	every script passed.
`1`	CLI usage error (unknown flag, missing script argument, …).
`2`	at least one script failed to transpile — never ran.
`3`	at least one script timed out (`-timeout`) or was context-cancelled.
`4`	at least one script ran and threw a JS exception.

`-v` writes lines prefixed with `[sercon]` to stderr. The traces include the full rewritten entry-script JS (the form goja actually runs, after the ESM→CJS rewrite + async IIFE wrapper) and every module-resolution event, so debugging an unexpected resolve target or a mis-rewritten import is straightforward.

The CLI registers nine reserved top-level globals; see the next section.

`--watch`: re-run on file change

`sercon --watch <script.ts>` runs the script once and then blocks, re-running it every time a watched file changes under the script root. Useful for iterating on a script body or on the

binding surface during development.

```
sercon --watch examples/scripts/smoke.ts
```

What's watched:

- The script root (resolved the same way as the one-shot mode — `-root DIR` if set, else `dirname` of the first script argument).
- Recursively. Symlinks aren't followed.
- New directories that appear during the session are picked up automatically.

What's filtered:

- **File extensions**: `.ts` / `.tsx` / `.js` / `.jsx` / `.json` trigger a re-run; `.d.ts` files match via suffix too. Anything else (Markdown, images, editor swap files, build artefacts) is ignored.
- **Directories**: hidden directories (`.git` , `.vscode` , anything starting with `.`) and `node_modules` are excluded — both because they generate floods of irrelevant events and because recursing into them inflates the watcher's directory count for no gain.

Re-runs are **debounced 150 ms**. Editors typically fire several events per save (write → rename → chmod); collapsing them into a single re-run keeps the loop responsive without thrashing. The debounce window resets on every event, so a rapid burst of saves becomes one re-run after the burst settles.

Each run is delimited by a line like `--- sercon re-run @ HH:MM:SS ---` so the output is visually distinct from the previous run.

`Ctrl-C` (SIGINT) or `SIGTERM` exit cleanly. Per-script failures inside a watch session log as `FAIL` but don't terminate the loop — a syntax error you're trying to fix shouldn't kick you out of watch mode.

The watcher reuses the same `Engine` across runs. Each `Run` already gets a fresh `*goja.Runtime` , so re-running is just re-invoking `runOne` on the existing engine — there's no per-iteration setup cost beyond the transpile + module-resolution work.

Module-graph invalidation. Watch mode tracks each entry script's import graph (its own file plus every module it resolves, captured via `Engine.SetResolveHook`) during the run. On a file change it re-runs **only the entries whose graph includes the changed file** — editing a helper that just one of three entry scripts imports re-runs that one entry, not all three. An entry's own file counts (editing it re-runs it); stdin entries and any entry whose graph isn't known yet re-run unconditionally (conservative). Before each re-run the module cache is busted (`Engine.ResetModuleCache`) so an edited import's new source actually takes effect — the registry otherwise caches compiled bytecode across runs.

5. Reserved globals (script surface)

sercon scripts get nine reserved top-level globals. Use them directly — there is no enclosing namespace. The full per-binding reference is the generated `examples/scripts/api.d.ts`; this section is the at-a-glance prose.

Reserved names: `runtime`, `crypto`, `text`, `codec`, `fs`, `net`, `db`, `services`, `tui`. User code can shadow these with a local `let` / `const` / `var` per normal JavaScript scoping — sercon does not intervene at runtime.

Migration from v0.8.0

v0.8.0	v0.9.0
`api.runtime.log`	`runtime.log`
`api.crypto.hash.sha256`	`crypto.hash.sha256`
`api.text.str.trim`	`text.str.trim`
`api.format.barcode.encode`	`codec.barcode.encode`
`api.fs.path.basename`	`fs.path.basename`
`api.net.http.get`	`net.http.get`
`api.db.sqlite.open`	`db.sqlite.open`
`api.tools.exec.shell`	`services.exec.shell`
`api.tools.git.commit`	`services.git.commit`
`api.ui.tui.layout`	`tui.layout`
`Sercon.argv`	`runtime.argv`

The `api` global no longer exists; reading it throws a `ReferenceError`. The `Sercon` global is also gone.

A mechanical sed pass over your scripts handles the prefix drop and the three renames — see the `CHANGELOG.md` `[Unreleased]` "Changed" entry for a sketch.

`runtime`

Script-host scaffolding. Members:

- `runtime.log(...args)` — stringify each argument and println the result.
- `runtime.assert.equal(actual, expected, msg?)` / `runtime.assert.ok(cond, msg?)` — throw on mismatch / falsy.
- `runtime.time.nowMs()` — current wall-clock in milliseconds.
- `runtime.time.sleep(ms)` — Promise that resolves after `ms` on the event loop.
- `runtime.time.format(unixMs, layout, tz?)` — Go-style time layout (e.g. `"2006-01-02 15:04:05"`).

- `runtime.env.get(name)` — process environment variable; returns `undefined` when unset (never throws).
- `runtime.argv` — `[programName, scriptPath, ...userArgs]`. User args come from the CLI args after a `--` separator. The layout mirrors Node/Bun: `argv[0]` is `"sercon"`, `argv[1]` is the path of the currently-executing script (so each script in a multi-arg invocation sees its own path), and `argv.slice(2)` is just your args. The property is always present; with no `--` it is just `[programName, scriptPath]`.

```
runtime.log("hello");
runtime.assert.equal(2 + 2, 4);
const args = runtime.argv.slice(2);   // post-`--` user args
```

`crypto`

Hashing, JWT, and age-style encryption. Members:

- `crypto.hash.*` — `md5`, `sha1`, `sha256`, `sha384`, `sha512`, `sha3_256`, `sha3_512`, `blake3`, `crc32`. All take a `string` and return the hex digest.
- `crypto.jwt.sign(claims, key, opts?)` / `crypto.jwt.view(token)` / `crypto.jwt.validate(token, key, opts?)` — HS256/RS256/ES256 JWTs.
- `crypto.encrypt.{keygen, keygenPgp, encrypt, decrypt, rekey, detectBackend}` — age-format symmetric encryption with PGP-compat key wrapping.

`text`

String, regex, charset, and data manipulation. Members:

- `text.str.*` — `trim`, `ltrim`, `rtrim`, `reverse`, `stripHtml`, `nl2br`, `br2nl`, `base64Encode`, `base64Decode`, `urlEncode`, `urlDecode`, `htmlEntityDecode`, `pad` / `lpad` / `rpad`, `sprintf`, `printf`, `normalizeNewlines`.
- `text.preg.*` and `text.preg2.*` — PCRE-style regex (PHP-compatible named-capture, lookbehind, etc.).
- `text.charset.detect(data)` / `decode(data, charset)` / `encode(text, charset)` — character-set detection and conversion.
- `text.jq.query(json, expr)` / `queryAll(json, expr)` — jq filtering against parsed JSON values.
- `text.diff.compare(a, b, opts?)` — unified / patience / inline diffs between two strings.

`codec`

Binary-format codecs (was `format` in v0.8.0). Members:

- `codec.compression.{algos, compress, decompress}` — gzip, deflate, zlib, bzip2, zstd, brotli, lz4, xz, snappy.

- `codec.barcode.{formats, decodableFormats, encode, decode}` — QR, DataMatrix, Aztec, PDF417, Code128, Code39, Codabar, EAN-13/8, UPC-A, ITF. `encode` returns PNG bytes; `decode` reads image bytes.
- `codec.checkdigit.{algos, validate, compute, inspect}` — Luhn, ISBN-10/13, EAN-13/8, UPC-A.

`fs`

Filesystem operations:

- `fs.path.dirname(p)` / `fs.path.basename(p, suffix?)` — POSIX-style.
- `fs.archive.create(srcPath, opts?)` / `fs.archive.extract(archivePath, destDir, opts?)` — tar/zip with optional compression.

`net`

Network clients and probes:

- `net.http.{get, post, request}` — fetch-style HTTP client; `request` accepts headers, body, timeout, retry, follow, and basic auth.
- `net.probe.{tcp, dns, tls, ntp, whois, ping, smtp, wss}` — one-shot reachability / handshake probes. Each returns a structured result; failures surface as `Error` objects with details.
- `net.netstatus.check(host)` — combined reachability summary.
- `net.email.*` — `spf`, `dmarc`, `mtasts`, `tlsrpt`, `bimi`, and `all(domain)` which runs every probe in parallel and aggregates.
- `net.browser.open(url)` — launches the OS default browser.

`db`

Database / KV / directory clients:

- `db.sqlite.open(path)` — embedded SQLite (cgo-free driver).
- `db.redis.open(addr, opts?)` — Redis client.
- `db.memcached.open(addr)` — Memcached client.
- `db.ldap.open(url, opts?)` — LDAP client (search, bind).
- `db.dict.{define, match}` — local dictionary lookup / fuzzy match.

`services`

Subprocess and external-CLI / service wrappers (was `tools` in v0.8.0). Members:

- `services.exec.shell(cmd, opts?)` — run a shell command; captures stdout/stderr or streams into a `tui` pane when `opts.pane` is set.

- `services.exec.http(method, url, opts?)` — shell-level `curl`-style HTTP (separate from `net.http`, intended for raw protocol use).
- `services.git.*` — git porcelain wrappers (clone, status, commit, log, …) invoking the local `git` binary.
- `services.gh.{authStatus, prList, repoView}` — thin `gh` CLI wrappers.
- `services.ai.{providers, send}` — provider-agnostic AI chat client.

`tui`

Multi-pane terminal UI (was `ui.tui` in v0.8.0). `tui.layout(tree)` declares panes; `tui.pane(name)` returns a pane handle with `write`, `writeln`, `clear`, `title`. `services.exec.shell({pane})` streams subprocess I/O into a pane.

Scripts that orchestrate multiple subprocesses (e.g. `brew`, `npm`, `cargo` updates running in parallel) can declare a multi-pane terminal layout and route each subprocess's stdout/stderr into its own pane. Activation is **script-driven**: the first call to `tui.layout(...)` enters TUI mode. If stdout is not a TTY (CI, pipes, `make demo`), the same calls fall back to prefixed plain-text lines (`[paneName] line`) so the same script runs in both contexts.

Layout

```
tui.layout({
  rows: [
    { name: "log", title: "Orchestrator", weight: 1 },
    { cols: [
        { name: "brew" },
        { name: "npm" },
      ],
      weight: 2,
    },
  ],
});
```

Each layout node is one of:

- `{ name: string, title?: string, weight?: number }` — a leaf pane.
- `{ rows: LayoutNode[], weight?: number }` — a vertical split.
- `{ cols: LayoutNode[], weight?: number }` — a horizontal split.

`weight` defaults to 1. Pane names must be unique across the whole tree. `layout` may be called **once** per Run; a second call throws.

Pane handles

```
const log = tui.pane("log");
log.writeln("Updating Homebrew…");
log.title("Done");
log.clear();
log.write("partial line ");
```

`tui.pane(name)` returns a handle with `write`, `writeln`, `clear`, `title`. Methods are synchronous from the script's perspective — they enqueue to the TUI goroutine.

Subprocess routing

`services.exec.shell(cmd, opts)` accepts `opts.pane` (a Pane handle or a pane name string):

```
await services.exec.shell("brew update && brew upgrade", { pane: "brew" });
```

When set, the subprocess's stdout **and** stderr stream into the pane line by line. The returned `{ exitCode, durationMs, success }` is the same as without `pane:` except `stdout` and `stderr` are empty (data was streamed, not captured). ANSI colors are translated to tview color tags and rendered natively; `\r` without `\n` overwrites the current line so progress spinners render cleanly.

Keybindings (TTY mode only)

Key	Action
`Tab` / `Shift-Tab`	Cycle focus through panes
`PgUp` / `PgDn`	Scroll focused pane one page
`↑` / `↓`	Scroll focused pane one line
`Home` / `End`	Jump to top / bottom
`Ctrl-C`	Abort the script

The focused pane has a yellow border; the bottom status bar shows its name and the active keys.

Limitations (v1)

- `tui` is **incompatible with `--watch`**: calling `tui.layout()` under `--watch` throws. Use one or the other.
- No mouse, no runtime layout mutation, no input panes (`tui.input`), no snapshot-to-normal-screen on exit. The alt screen is restored at script end and the usual `PASS` / `FAIL` line prints.

6. JavaScript runtime built-ins (goja)

`scriptengine` runs on goja, which implements **ES5.1 + a large subset of ES6+**. The following are present and behave per spec:

Globals

- `Object`, `Array`, `String`, `Number`, `Boolean`, `BigInt`
- `Math`, `Date`, `JSON`, `RegExp`
- `Error`, `TypeError`, `RangeError`, `SyntaxError`, `ReferenceError`, `EvalError`, `URIError`
- `Promise` (provided via goja's native implementation; resolution microtasks are pumped by the event loop)
- `Symbol` (partial: well-known symbols `Symbol.iterator`, `Symbol.asyncIterator`, etc., are supported)
- `Proxy`, `Reflect` (partial)

```
runtime.log(Math.PI.toFixed(4), Math.max(1, 9, 3));
runtime.log(new Date().toISOString());
runtime.log(JSON.stringify({ a: 1, b: [2, 3] }));
runtime.log("abc".repeat(3), "abc".padStart(6, "_"));
```

Collections

- `Map`, `Set`, `WeakMap`, `WeakSet`

```
const counts = new Map<string, number>();
for (const ch of "banana") counts.set(ch, (counts.get(ch) ?? 0) + 1);
runtime.log([...counts]); // [["b",1],["a",3],["n",2]]
```

Typed arrays

- `ArrayBuffer`, `DataView`
- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `Float32Array`, `Float64Array`

```
const buf = new ArrayBuffer(4);
new DataView(buf).setUint32(0, 0xCAFEBABE);
runtime.log(new Uint8Array(buf)[0].toString(16)); // ca
```

Iteration and generators

- `for…of`, `for…in`, spread, destructuring

- Generator functions (`function*`)
- Async generators (`async function*`) — supported via goja's event loop

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const it = fib();
const first10 = Array.from({ length: 10 }, () => it.next().value);
runtime.log(first10);
```

Not (yet) provided

- `fetch` — use `net.http.*` from the CLI, or register your own binding.
- `Worker` , threading primitives.
- DOM globals (`window` , `document`).
- Native ES modules (`import` / `export` are *transpiled* to CommonJS at load time — see section 8).

7. Async runtime additions (goja_nodejs)

The event loop bundles a small Node-compatible runtime on top of goja:

Timers

```
setTimeout(() => runtime.log("after 50ms"), 50);
setInterval(() => runtime.log("tick"), 100);
setImmediate(() => runtime.log("next tick"));
clearTimeout(t); clearInterval(i); clearImmediate(im);
```

The event loop holds the script alive while *any* timer is pending. If your script ends with only `setTimeout` callbacks scheduled, the run will wait for them to fire before returning.

`console`

```
console.log("info");
console.error("oops");
console.warn("careful");
console.info("fyi");
console.debug("trace");
```

Disable with `Options.DisableConsole = true` if you want a clean global namespace.

`require`

CommonJS require, with TS-aware extension fallback added by sercon (see section 10):

```
const helpers = require("./helpers");
helpers.doThing();
```

Both `require("./foo")` and `import { x } from "./foo"` resolve to the same module instance per Run; esbuild rewrites `import` to `require` at transpile time.

8. TypeScript support

TS is transpiled by esbuild in-process. There is no `tsc`, no `tsconfig.json`, no type-checking — types are erased and the resulting JS is what executes. Practical implications:

- All TS syntax esbuild supports is allowed: type annotations, `interface`, `type`, generics, enums (compiled to objects), namespaces, decorators (legacy & TC39 stage 3).
- `import type { X } from "y"` is stripped entirely (no runtime require).
- TS errors are *not* surfaced as build errors — only esbuild parse errors are. Run a separate `tsc --noEmit` in CI if you want type enforcement.
- `tsconfig.json` is **not** consulted.

`.tsx` and `.jsx` are supported via esbuild's `LoaderTSX` / `LoaderJSX`, including for required modules resolved by the engine's extension fallback. JSX has no React runtime in scope by default — either set the factory at the file level with an `@jsx` pragma:

```
/** @jsx h */
function h(tag: string, props: any, ...children: any[]) {
  return { tag, props: props ?? {}, children };
}
export const Box = (label: string) => <div className="box">{label}</div>;
```

…or provide a `React.createElement`-compatible binding from Go and rely on esbuild's default pragma. ESM `export default <value>` also works through the entry-script rewriter's `__esModule ? .default : m` unwrap, so `import answer from "./mod"` resolves to the default export.

9. Top-level `await`

Top-level `await` works in entry scripts:

```
const r = await net.http.get("https://example.com");
runtime.log(r.status);
```

How it works under the hood: esbuild rejects top-level await with the `cjs` output format, so the engine transpiles the *entry* script with `format=esm`, then rewrites `import` statements to `require()` calls and wraps the rest of the body in:

```
;(async () => { /* your transpiled body */ })().then(__resolve, __reject);
```

`__resolve` and `__reject` are bindings the engine sets on the runtime to capture the final settlement. *Required-module* code (anything loaded through `require`) is transpiled with `format=cjs` and does **not** support top-level await — wrap in `(async () => …)()` yourself if you need it inside a module.

10. Module resolution

The engine plugs a custom source loader into `goja_nodejs/require` that adds:

1. Direct path: if the requested file exists on disk, use it. `.ts` / `.tsx` files are transpiled on the fly.
2. **Extension fallback** when the request has no extension: try `.ts`, `.tsx`, `.js`, `.cjs`, `.mjs`, `.json` in that order.
3. **`.js` → `.ts` swap**: if a path ending in `.js` / `.cjs` / `.mjs` is requested but the literal file doesn't exist, the same path ending in `.ts` (or `.tsx`) is tried. Handles `package.json` `main` fields that point at compiled output where only the TypeScript source is on disk.
4. **`package.json` `source` preference**: when reading a `package.json`, if a `source` field is present and points at an existing `.ts` / `.tsx` file, `main` is rewritten to that path before the registry consumes it. Matches the convention used by parcel, microbundle, and similar bundlers to mark the TS source of truth.
5. **Directory index**: if the resolved path is a directory, try `index.ts`, `index.tsx`, `index.js`.

The base directory follows Node's CommonJS rules — relative imports resolve against the requiring module's directory, courtesy of goja_nodejs.

```
// All resolve to ./helpers/assert.ts:
import { check } from "./helpers/assert";
import { check } from "./helpers/assert.ts";
const { check } = require("./helpers/assert");
const { check } = require("./helpers/assert.js");
```

JSON imports work out of the box via either the ESM-style default import (esbuild rewrites it to a require) or a direct `require`:

```
import data from "./data.json";
const r = require("./data.json");
```

`node_modules` lookup *works* via the upstream registry, but the source loader doesn't currently transpile through that path — keep TS helpers under `ScriptRoot`.

11. Timeouts and cancellation

Two mechanisms interrupt a running script:

- `Options.Timeout` — wall-clock limit per `Run`. Exceeding it returns `scriptengine.ErrScriptTimeout`.
- `ctx` passed to `Run` / `RunFile` — cancelling the context returns `ctx.Err()` (typically `context.Canceled` or `context.DeadlineExceeded`).

Both call `vm.Interrupt(...)` and `loop.Terminate()` from a watcher goroutine. This works for **sync** JS — including `while(true){}` — because the goja VM checks the interrupt flag between bytecode steps. A host Go binding that's blocked in cgo or a long syscall isn't interruptible mid-call; if you write such a binding, plumb the engine `ctx` through to it yourself.

```
eng := scriptengine.New(scriptengine.Options{Timeout: 200 *
time.Millisecond})
_, err := eng.Run(ctx, "loop.ts", `while (true) {}`)
// err is scriptengine.ErrScriptTimeout, returned ~200–300ms after Run.
```

12. Error semantics

- A Go binding returning `(T, error)` automatically throws as a JS exception when `err != nil`. The exception's `.message` is `err.Error()`.

  ```
  try { mayFail(); } catch (e) { runtime.log(String(e)); }
  ```

- A Go binding that `panic`s with a `*goja.Object` (e.g. `vm.NewGoError(...)`) does the same.
- A script throwing a JS error out of the top level surfaces from `Run` as a Go `error` whose message is the JS error's `.message`.
- Timeout errors satisfy `errors.Is(err, scriptengine.ErrScriptTimeout)`. Cancellation errors satisfy `errors.Is(err, context.Canceled)` or `context.DeadlineExceeded`.

13. Type generation (.d.ts)

`Engine.WriteTypes(w)` walks the registered bindings and emits a TypeScript declaration file. The mapping table (abbreviated):

Go type	TS type
`string`	`string`
any numeric / `float64`	`number`
`bool`	`boolean`
`[]T`	`T[]` ; `[]byte` → `Uint8Array`
`map[string]T`	`Record<string, T>`
`*T`	`T` (transparent unwrap)
`error` (single return)	omitted (collapses to `void`)
`(T, error)` (tuple return)	`T`
`interface{}` / `any`	`unknown`
`goja.FunctionCall` parameter	folded into `(...args: unknown[])`
`goja.Value` return	`unknown`
`scriptengine.AsyncBinding` value (from `PromisifyAsync[T]`)	`Promise<T>`
self-referential types	`unknown` (cycle detection bails after depth 4)

```
sercon --emit-dts d.ts
```

In your editor, place the resulting `.d.ts` next to your scripts (or reference it via `/// <reference path="..." />`) for autocomplete.

JSDoc comments on declarations

The emitter pulls JSDoc strings from the engine's doc map (set via `Engine.SetDocs` / `Engine.SetMemberDocs`) and renders them above each declaration. Single-line docs collapse to `/** … */`; multi-line docs expand to a standard `*`-prefixed block. Bindings without a doc entry emit no JSDoc — the emitter never inserts empty placeholders. Docs are looked up by the same dotted path the binding was registered under, so namespace members get hover text too:

```
// AUTO-GENERATED by scriptengine.Engine.WriteTypes — do not edit by hand.

/** Hashing primitives. */
declare const crypto: {
  hash: {
    /** SHA-256 hex digest of a UTF-8 input. */
    sha256(...args: unknown[]): string;
    /** BLAKE3 hex digest (32-byte output, lukechampine.com/blake3). */
    blake3(...args: unknown[]): string;
    // …
  };
  // …
};
```

The CLI's reserved-global surface ships with a curated doc map; see `cmd/sercon/api_docs.go` for the source of truth and add new entries there when adding new bindings (the lockstep rule keeps `--examples` / MANUAL / `d.ts` in sync, and the doc map is now the seventh artifact in that chain).

14. Limitations and gotchas

- **No native ES modules at runtime.** All `import` is rewritten to `require` by esbuild. Side-effect-only imports run the module body exactly once per Run.
- **No top-level await inside required modules.** Wrap with `(async () => … )()` inside the module if you need it.
- **No `tsconfig.json` honoured.** Module resolution and type checking are independent from any project-level TS config.
- **Per-Run runtime.** Side effects on `globalThis` do not persist between calls to `Run` on the same `Engine`. The require *compile* cache is shared; module *exports* are not.
- **Multi-line ESM imports may stress the entry rewriter.** The current rewriter is line-based and handles the common forms; especially gnarly inputs (comments inside the spec list, very unusual whitespace) fall back to executing as-is, which may throw.
- **`RegisterConstructor` is d.ts-only today.** At runtime it behaves like `Register`. True `new`-able constructor semantics are on the roadmap.
- **HTTP bindings are real network calls.** `net.http.*` uses `net/http` with a 5s timeout. They are not mockable from JS.

See OUT-OF-SCOPE.md for the active backlog of deferred ideas.

This manual covers sercon v0.8.0. Whenever you add, remove, or change a flag, a binding, or the script API, update this file alongside the help screen (`--help`), the examples walkthrough (`--examples`), and the `CHANGELOG.md`.